STUDENT RECORD AND ACADEMIC MANAGEMENT

Project Output Screenshots with Simple Theory

Project overview

This Python menu-driven program manages student records. The user chooses an option from a menu to add, view, search, update, or delete records and to calculate simple academic summaries. The screenshots below are arranged in menu order, with each unique screenshot included once.

Main concepts used

• **Input:** `input()` collects choices and student details from the user.

• **Conditions:** `if/elif/else` can select which menu operation to run.

• **Loops:** a loop can repeatedly show the menu until the user chooses Exit.

• **Lists and dictionaries:** commonly used to store multiple student records and each student's fields.

• **Functions:** separate functions make each operation easier to understand, test, and maintain.

• **File handling:** records are saved so they can be available after the program closes.

Example record shown in the screenshots

The example contains one student record. The department is AIDS, the subjects shown are Python and English, marks are 98, and attendance is 96%. Optional fields such as registration number and address are blank in the displayed example.

1. Add Student

Adds a new student record by collecting details such as roll number, name, department, date of birth, contact information, subjects, marks, and attendance. The program confirms that the record has been saved.

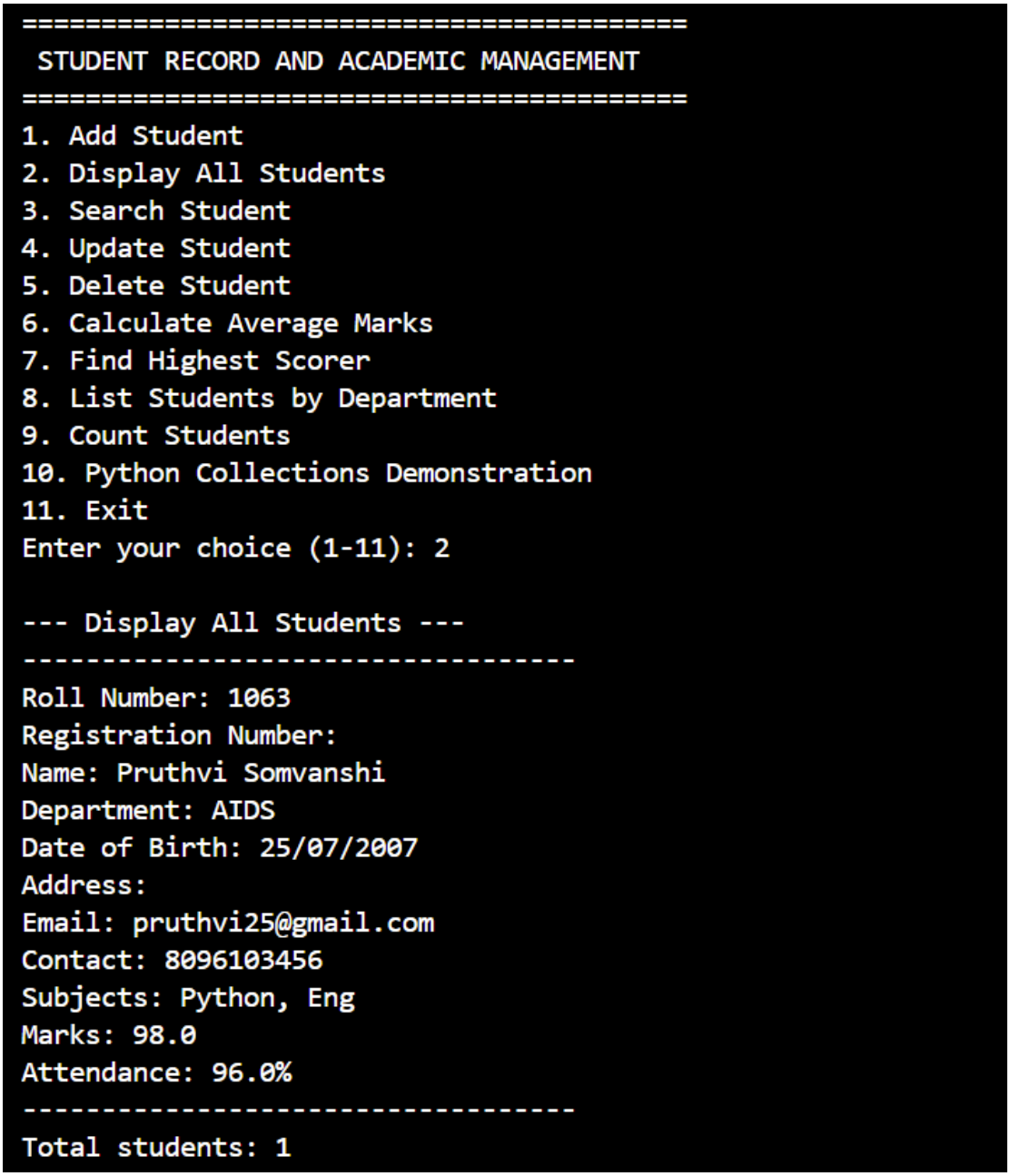
```
========================================
 STUDENT RECORD AND ACADEMIC MANAGEMENT
========================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Calculate Average Marks
7. Find Highest Scorer
8. List Students by Department
9. Count Students
10. Python Collections Demonstration
11. Exit
Enter your choice (1-11): 1

--- Add Student ---
Enter Roll Number: 1063
Enter Student Name: Pruthvi Somvanshi
Enter Department: AIDS
Enter Registration Number (or press Enter to skip):
Enter Date of Birth (or press Enter to skip): 25/07/2007
Enter Address (or press Enter to skip):
Enter Email ID (or press Enter to skip): pruthvi25@gmail.com
Enter Contact Number (or press Enter to skip): 8096103456
Enter Subjects separated by commas: Python,Eng
Enter marks (0-100): 98
Enter attendance percentage (0-100): 96
Student added and saved successfully!
```

Screenshot 1 of 11

2. Display All Students

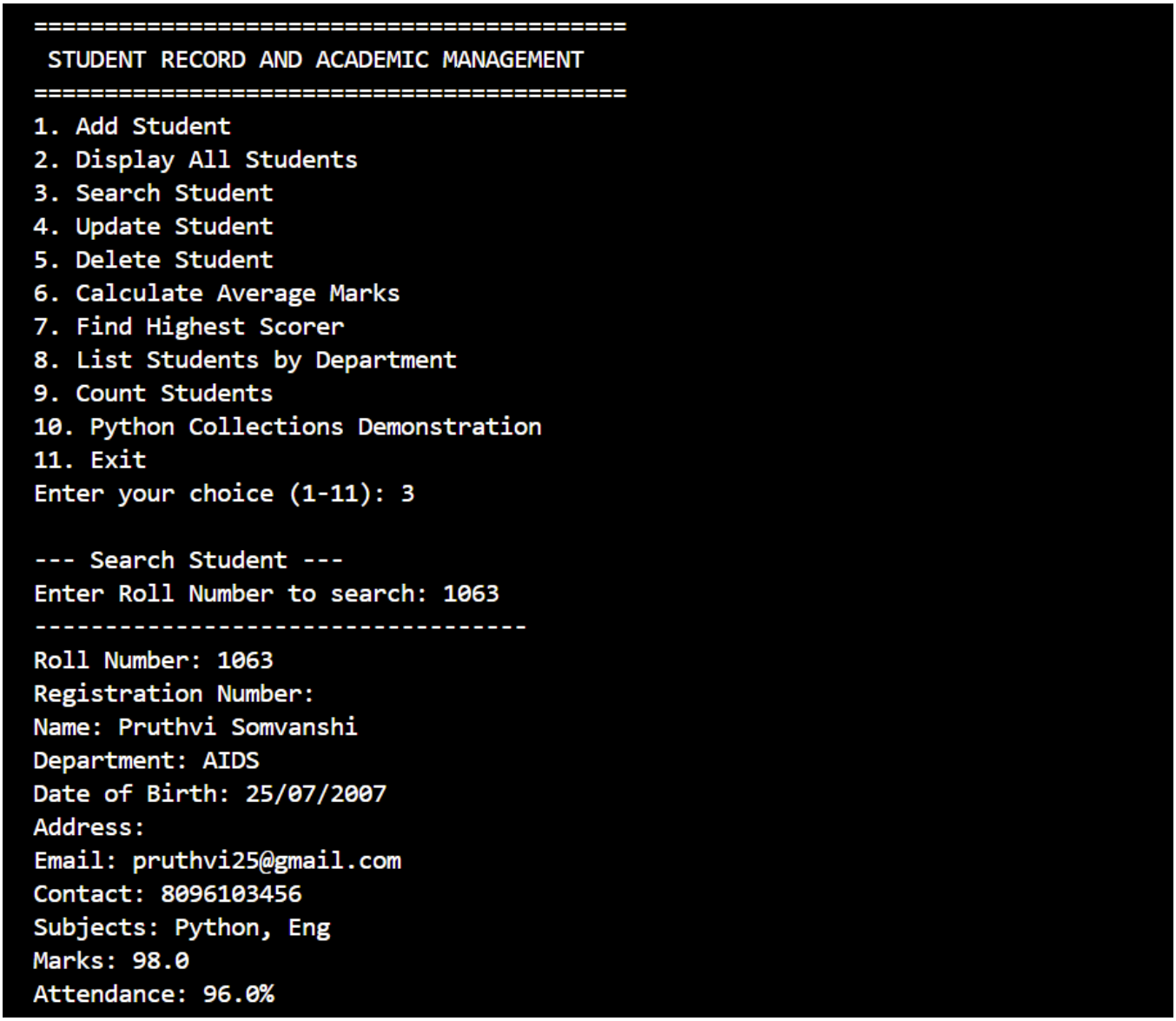
Displays every saved student record in a readable format. It is useful for checking the stored details and confirming how many student records exist.



Screenshot 2 of 11

3. Search Student

Searches for a student using the roll number. If a matching record exists, the program displays the student's information; a roll number works as a unique identifier in this example.



Screenshot 3 of 11

4. Update Student

Allows existing student details to be edited. Pressing Enter keeps the current value; optional fields can be left unchanged. The program then saves the updated record.

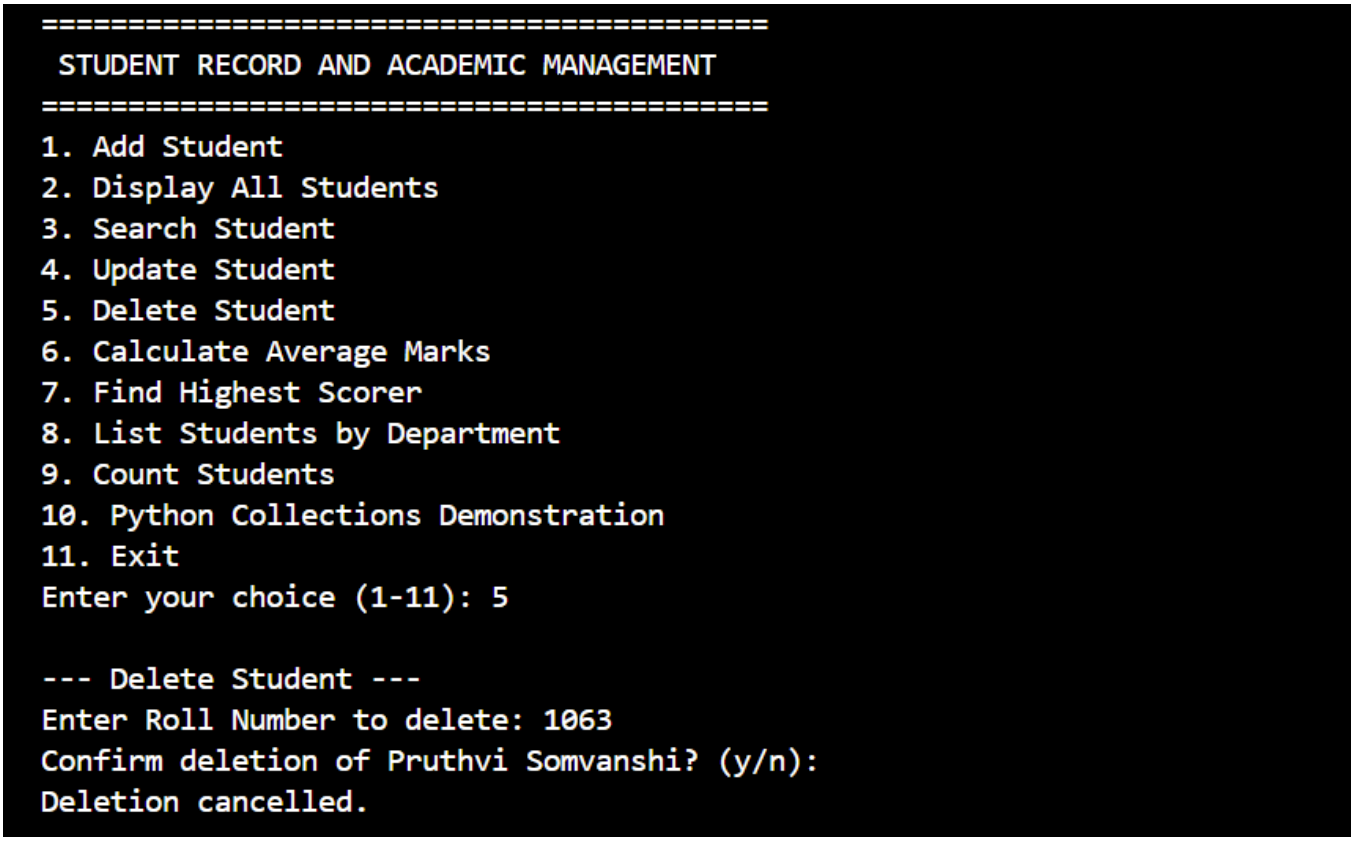
```
========================================
 STUDENT RECORD AND ACADEMIC MANAGEMENT
========================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Calculate Average Marks
7. Find Highest Scorer
8. List Students by Department
9. Count Students
10. Python Collections Demonstration
11. Exit
Enter your choice (1-11): 4

--- Update Student ---
Enter Roll Number to update: 1063
Press Enter to keep the existing value.
Name [Pruthvi Somvanshi]:
Department [AIDS]:
Registration Number []:
Date of Birth [25/07/2007]:
Address []:
Email ID [pruthvi25@gmail.com]:
Contact Number [8096103456]:
Update subjects (comma-separated, or Enter to keep current):
Update marks? (y/n):
Update attendance? (y/n):
Student record updated and saved.
```

Screenshot 4 of 11

5. Delete Student

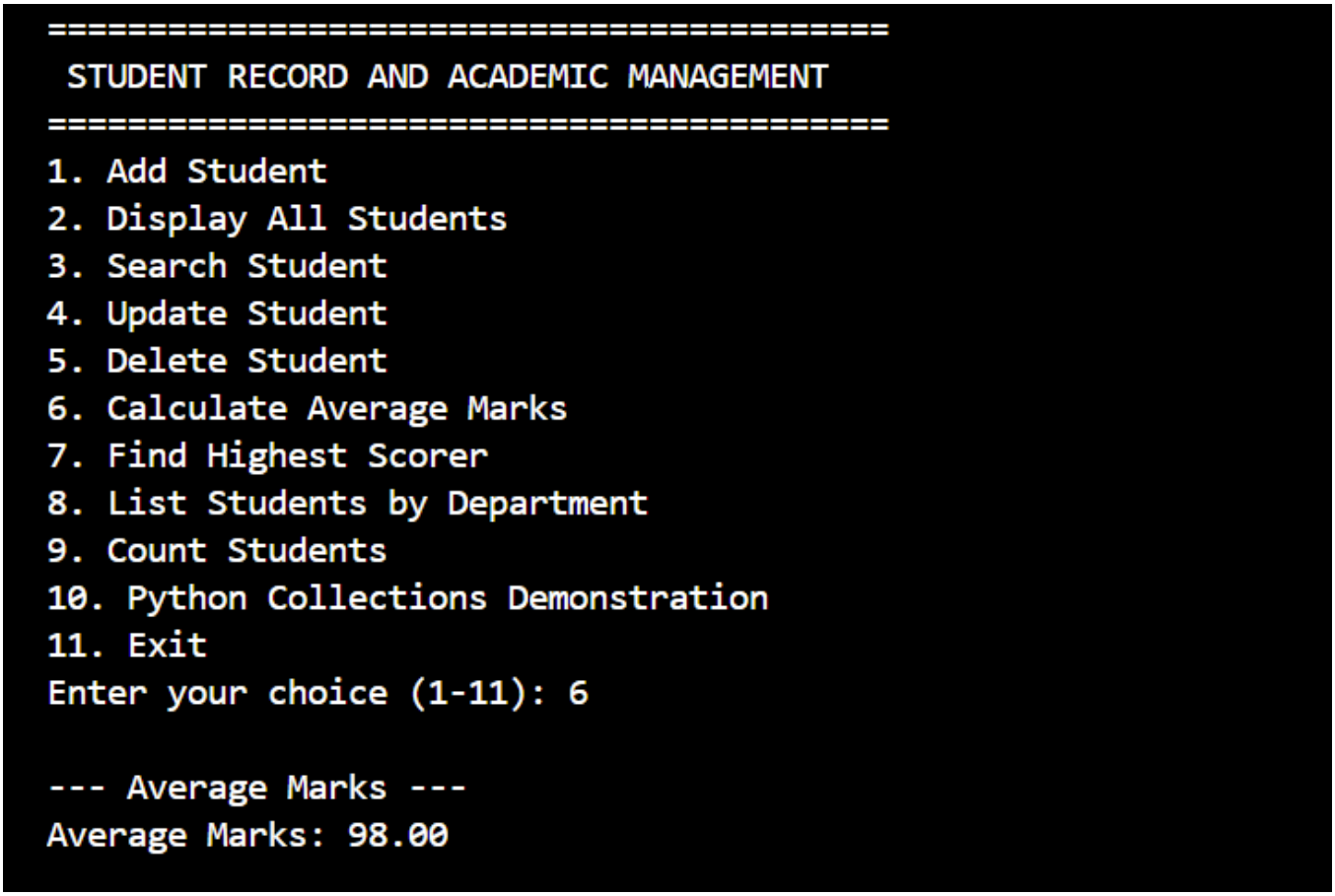
Removes a student record only after confirmation. In this screenshot, deletion is cancelled, so the record is not removed. This demonstrates a safety check before a destructive action.



Screenshot 5 of 11

6. Calculate Average Marks

Calculates the average of the marks stored for the student records. The displayed average is 98.00 because the example currently contains one student with marks of 98.



Screenshot 6 of 11

7. Find Highest Scorer

Finds and displays the student with the highest marks among the saved records. With only one record in the example, that student is the highest scorer.

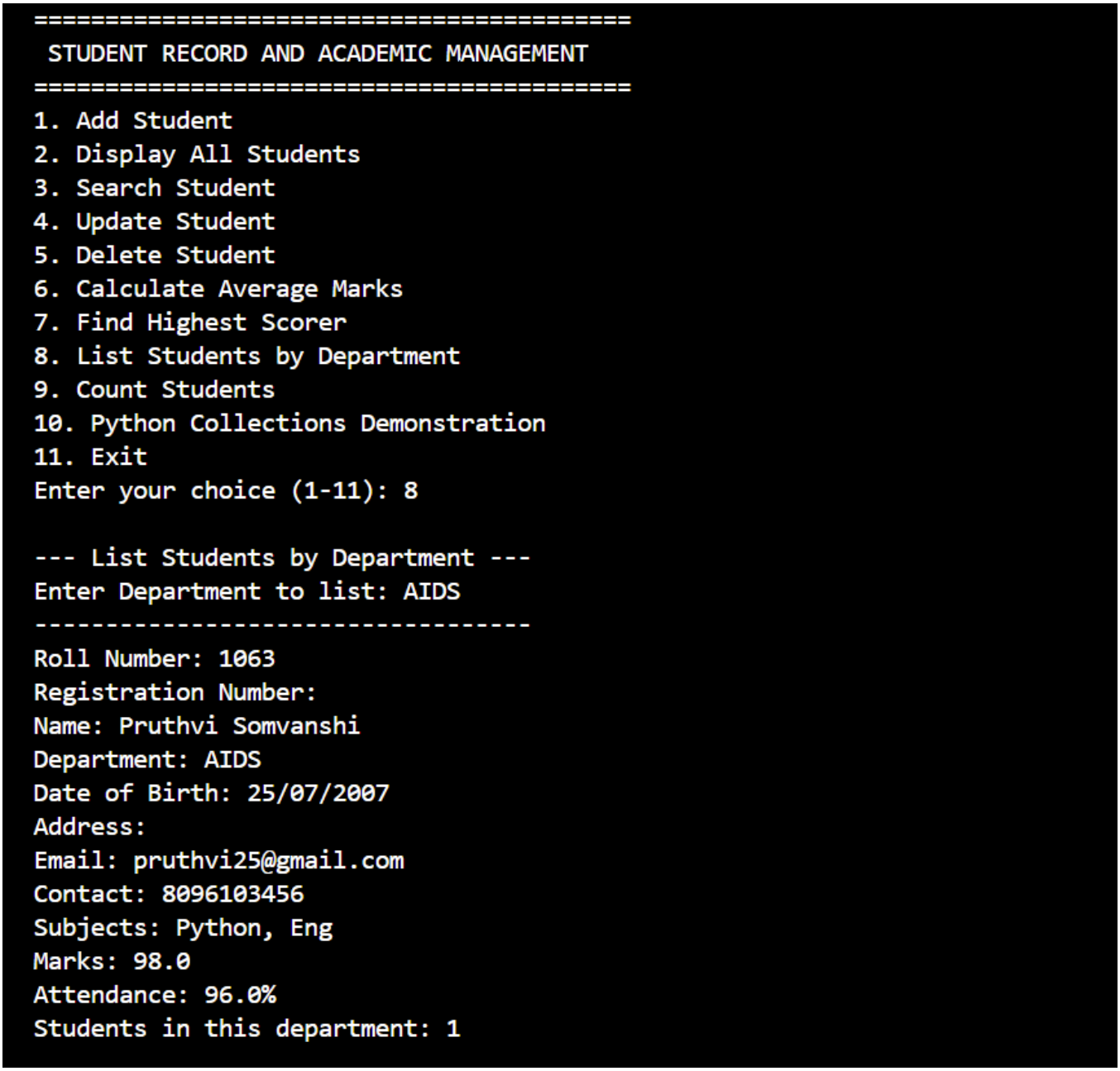
```
==========================================
 STUDENT RECORD AND ACADEMIC MANAGEMENT
==========================================
1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Calculate Average Marks
7. Find Highest Scorer
8. List Students by Department
9. Count Students
10. Python Collections Demonstration
11. Exit
Enter your choice (1-11): 7

--- Highest Scorer ---
----------------------------------
Roll Number: 1063
Registration Number:
Name: Pruthvi Somvanshi
Department: AIDS
Date of Birth: 25/07/2007
Address:
Email: pruthvi25@gmail.com
Contact: 8096103456
Subjects: Python, Eng
Marks: 98.0
Attendance: 96.0%
```

Screenshot 7 of 11

8. List Students by Department

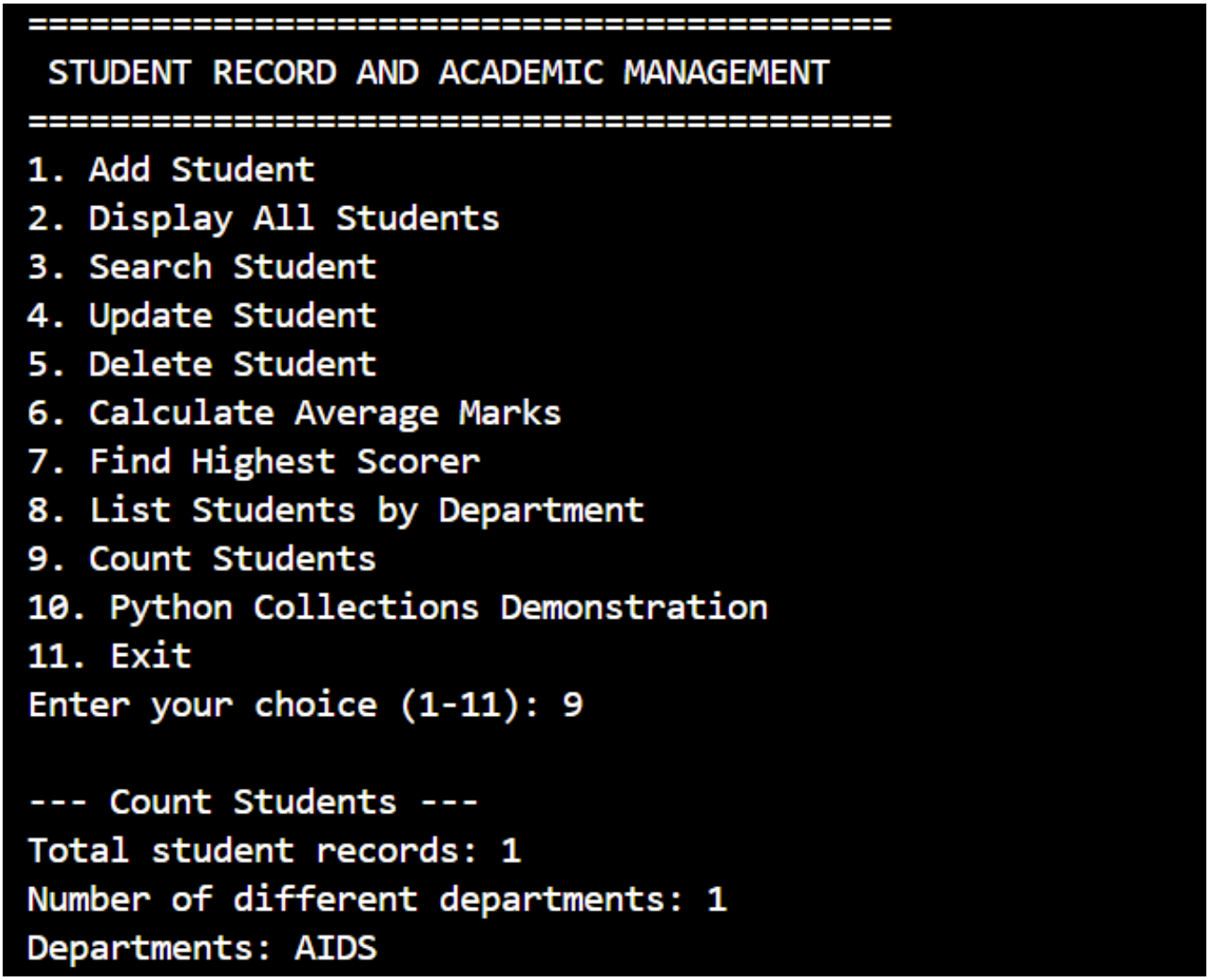
Filters records by department name. Entering AIDS displays the student whose department is AIDS and shows the number of matching students.



Screenshot 8 of 11

9. Count Students

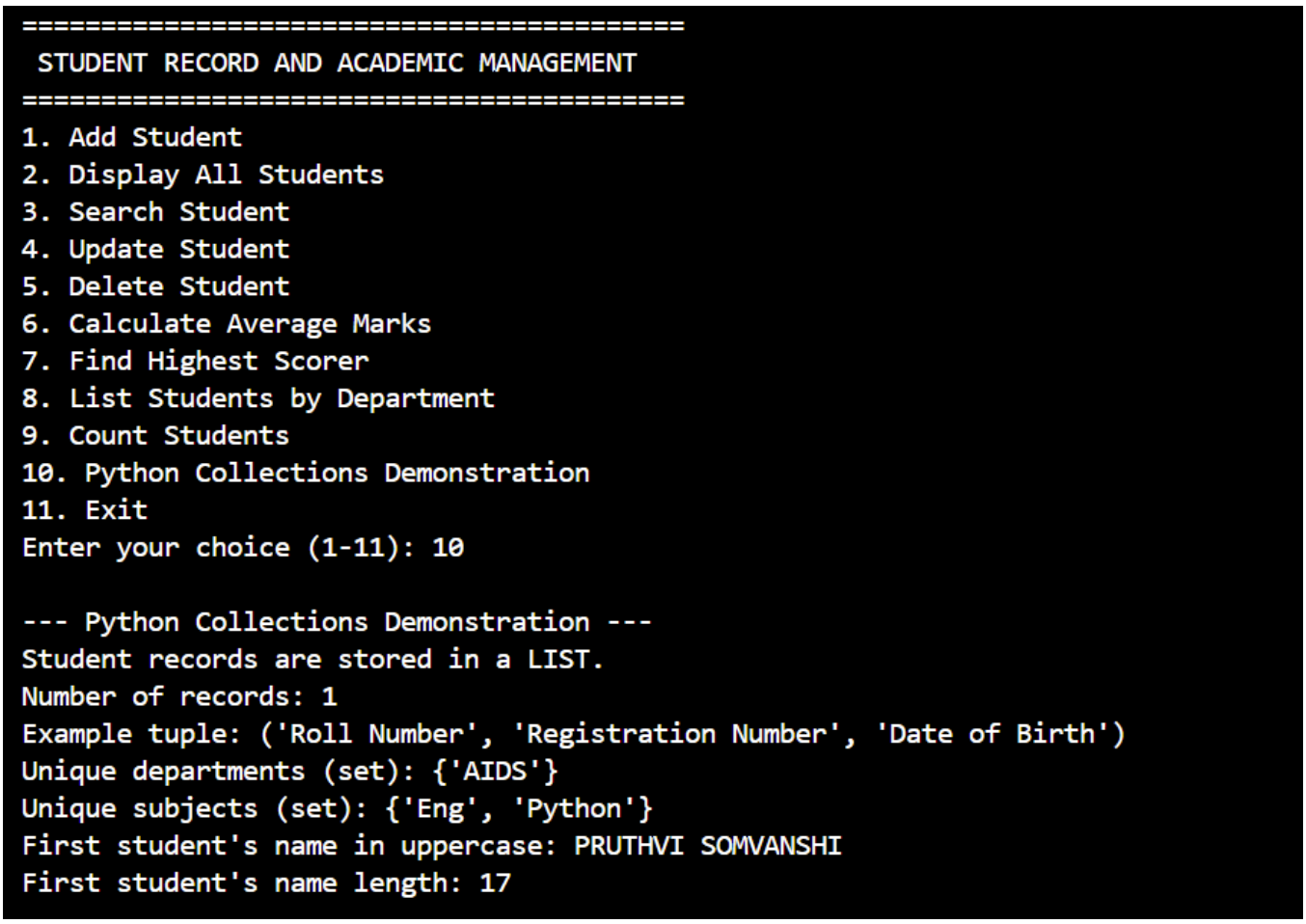
Counts the total number of student records and the number of different departments. The screenshot shows one student record and one department: AIDS.



Screenshot 9 of 11

10. Python Collections Demonstration

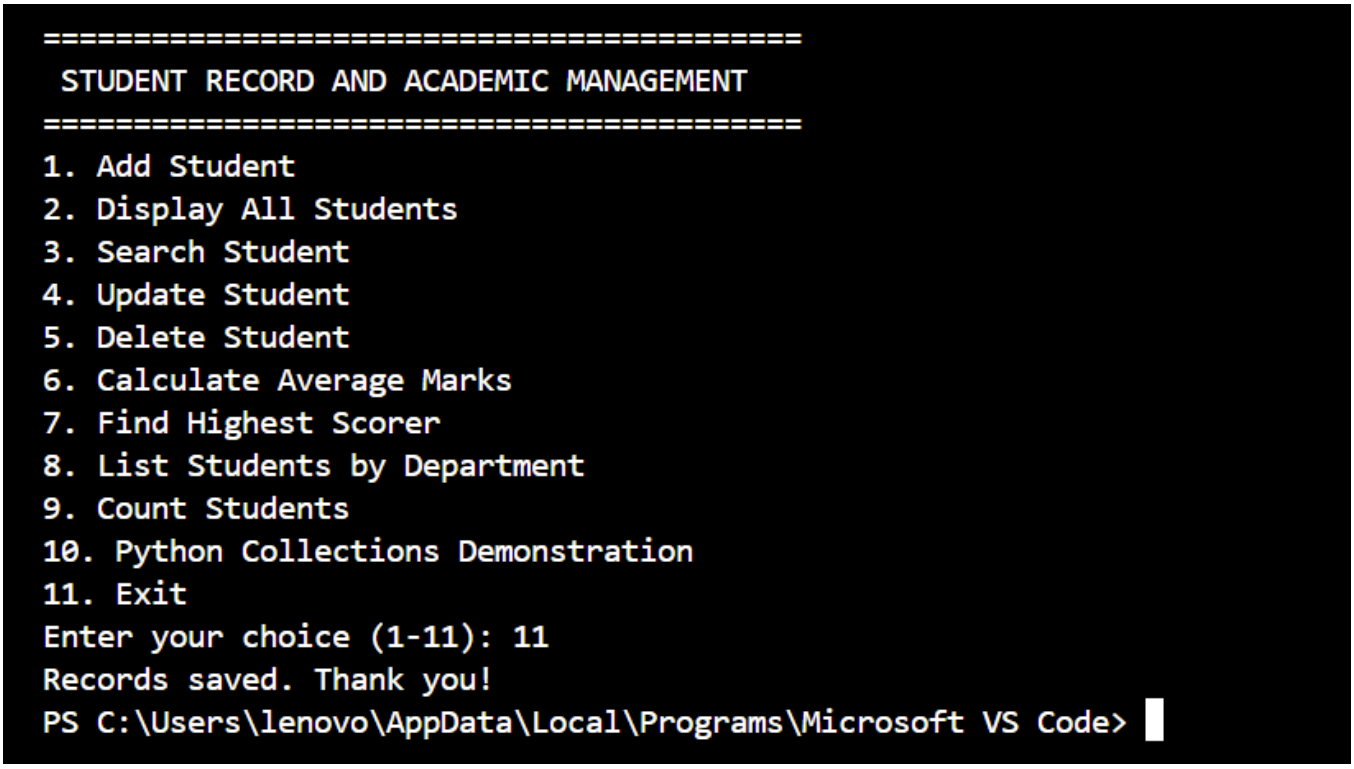
Demonstrates Python collection and string operations. The output indicates that records are stored in a list, a tuple is used as an example of grouped fields, and sets are used to show unique departments and subjects. It also converts the student's name to uppercase and measures its length.



Screenshot 10 of 11

11. Exit

Choosing option 11 ends the program. The message confirms that records were saved before the application returned to the terminal prompt.



Screenshot 11 of 11

Conclusion

The output demonstrates the main functions of a student record management system: creating records, viewing and finding records, editing or removing records, and calculating summaries. The Python collections demonstration also shows how lists, tuples, and sets can help organise and analyse information. The Exit option closes the program after saving the records.

Important note

The screenshots document one sample run and one student record. Results such as the average marks and highest scorer can change when more students are added or their marks are updated.